

WEEK 1

Frequency Throttling Side Channel Attack

SUPERVISORS

Prof Abhishek Bichhawatt
Subhrajit Das

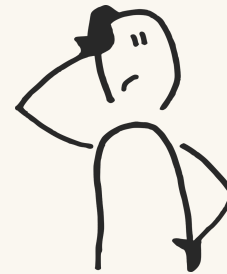
Ramji Purwar
24110287

Project Overview : Why it matters



Traditional Side Channel Attacks

Make the victim execute same number of cycle for every secret value. This removes ordinary data-dependent timing difference.



Hidden Assumption

One cycle take the same amount of wall-clock time. DVFS and reactive throttling breaks this assumption.



Paper Results

A constant cycle AES-NI workload can still have secret dependent runtime after frequency throttling.

Leakage Path

Secret Data

Data Dependent Power

Power Limit Hit

Lower freq

Longer Time

CPU power management background

DVFS / P-States

CPU voltage and frequency are adjusted to balance performance, energy, electrical limits, and thermal limits.

Reactive limit

The controller tracks running averages such as power or current over a time window τ .

Throttle

When a limit is exceeded, the controller lowers the maximum allowed frequency $f_{\max}$.

Conceptual controller

Δ = measured average – reactive limit

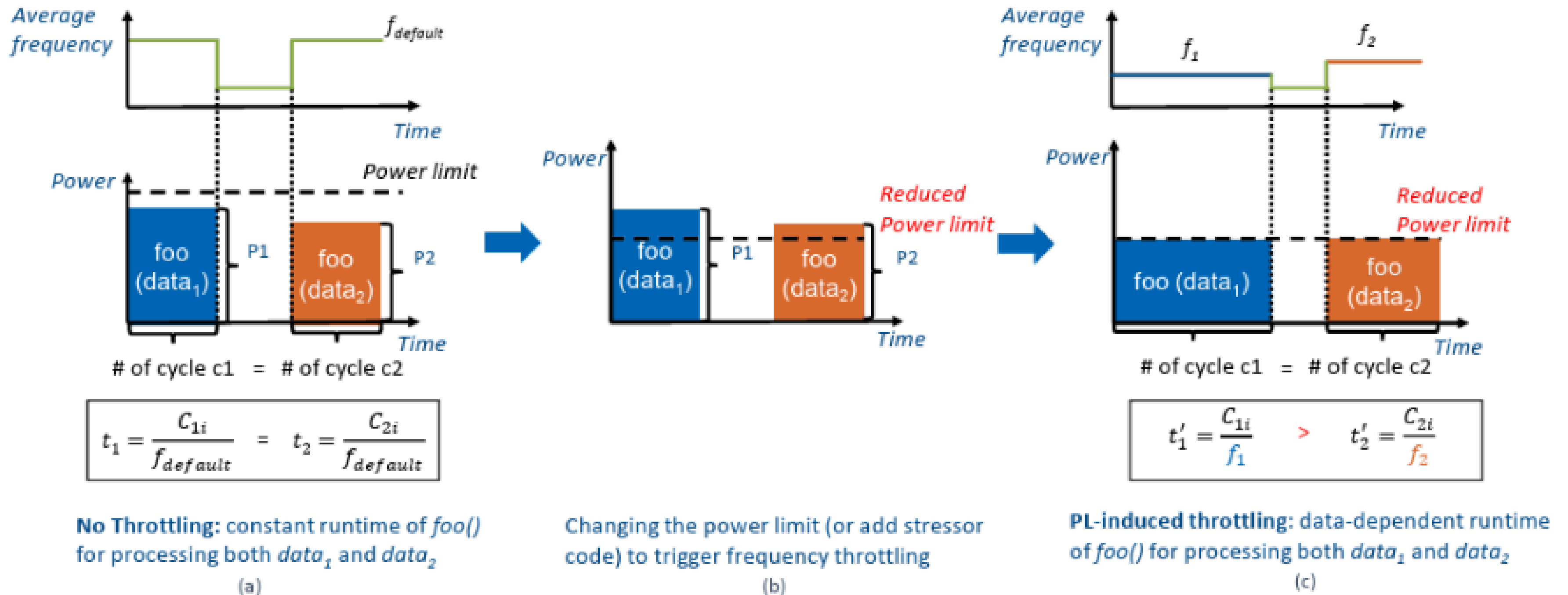
$f_{\max} = \text{PL_ALG}(\Delta)$

If Δ crosses the limit $\rightarrow$ throttling $\rightarrow$ lower frequency

Examples in the paper

Intel: PL1 / PL2 and VR-TDC current limit
AMD: PPT / PPT Fast and TDC
Shorter τ makes attacks faster because traces can be shorter.

The core attack mechanism



Proof-of-concept: the leak becomes measurable

Frequency Throttling Side-Channel Attack

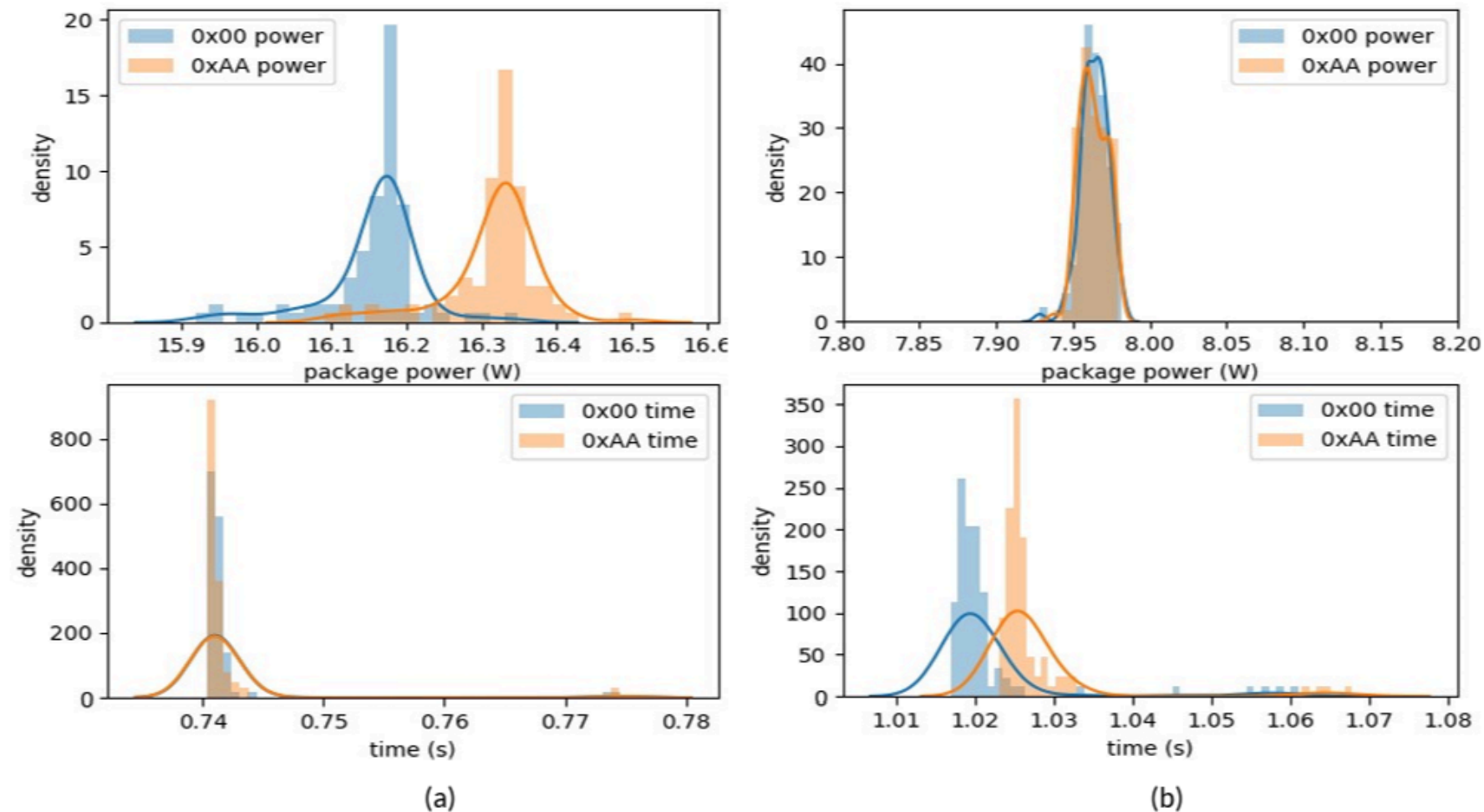


Figure 2: Average power consumption and time elapsed of the same IMUL workload with different operands (a) when throttling is not triggered, and (b) when there is reactive limit induced throttling.

Experiment A

- High reactive limits
- no throttling
- different operands consume different power but runtime is identical.

Experiment B

- PL1 reduced to 8 W, $\tau = 1$ s
- throttling is triggered.

Observed effect

- 0xAA consumes more power than 0x00.
Under throttling, the power distributions are capped near the limit, but 0xAA takes longer to execute.

Threat model and attack workflow

TWO ATTACKER CAPABILITIES ARE EVALUATED

Scenario 1 — Privileged attacker (Ring 0 / hypervisor)



Victim may run inside a TEE such as SGX or SEV. Attacker can configure reactive limits to deliberately trigger throttling.

Scenario 2 — User-space attacker (Ring 3)

Cannot configure limits. Instead, runs a stressor workload in parallel to push system power over an existing limit. This is noisier and harder.



1 CONFIGURATION

Profile victim-like workload; tune the limit or find a stressor.

2 ONLINE

Send inputs; measure victim wall-clock time with timestamps.

3 ANALYSIS

Use TVLA to detect leakage, then CPA to recover key bytes.

AES-128 case study

Why AES-NI is a strong test: the implementation is designed to be constant-cycle

Victim

AES-128 encryption using AES-NI instructions. The paper targets a fixed secret key while varying plaintexts.

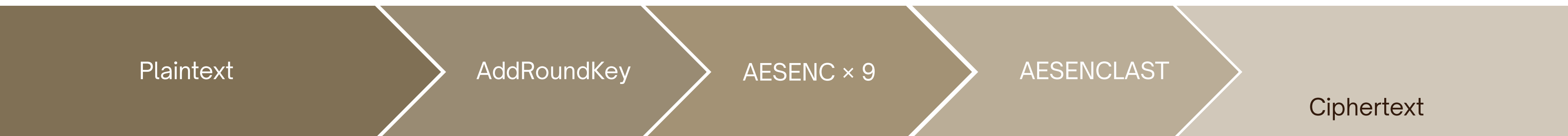
Why it should look safe

AES-NI accelerates AES using dedicated instructions and provides constant-cycle behavior intended to reduce timing leakage.

Attack signal

Secret-dependent internal switching changes power. Throttling turns that difference into a measurable execution-time difference.

Simplified attack point



CPA models: Round0-HW, Round10-HW, Round10-HD, Round10-HW+HD

How the authors extract the key

Workflow Different plaintexts → timing traces → statistical leakage test → key hypothesis ranking → recovered bytes

TVLA / Welch's t-test

$|t| > 4.5$ is used as the threshold for statistically distinguishable trace sets (99.999% confidence in the paper's setup).

CPA

For each key guess k , correlate measured timing T_δ with a hypothetical leakage model derived from Hamming weight or distance.

Guessing Entropy

Rank correct key-byte guesses by correlation. GE = 0 means all 16 AES key bytes have rank 1.

Welch t-score

$$t\text{-score} = \frac{\mu_1 - \mu_2}{\sqrt{s_1^2/N_1 + s_2^2/N_2}}$$

CPA correlation

$$\gamma^k = \frac{\text{cov}(T_\delta, T_h^k)}{\sigma_{T_\delta} \sigma_{T_h^k}}$$

GE

$$\text{GE} = \sum_{i=1}^{16} \log_2[\text{rank}(k_i^{\text{correct_key}})]$$

Results

Converged GE (lower is better)

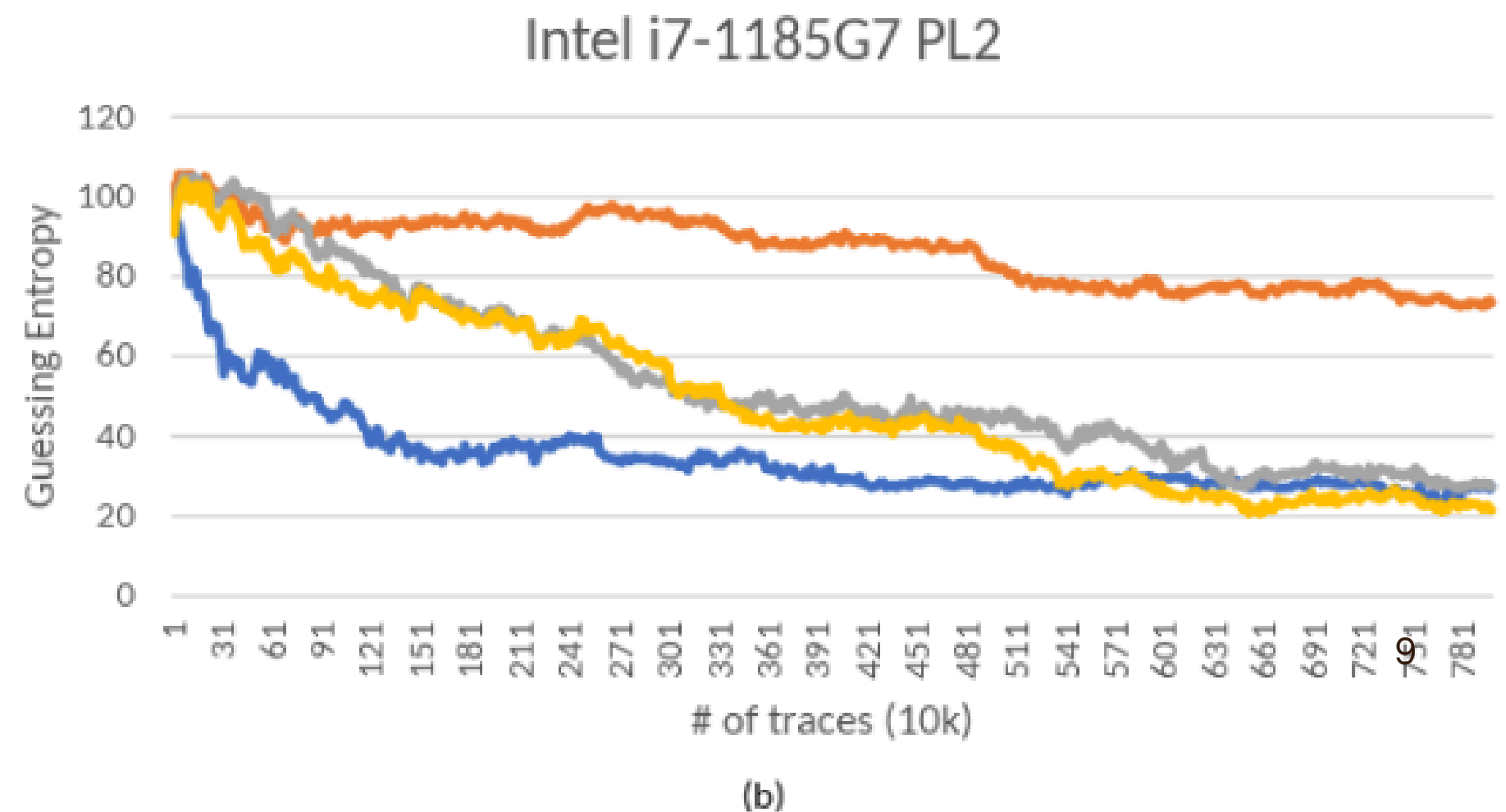
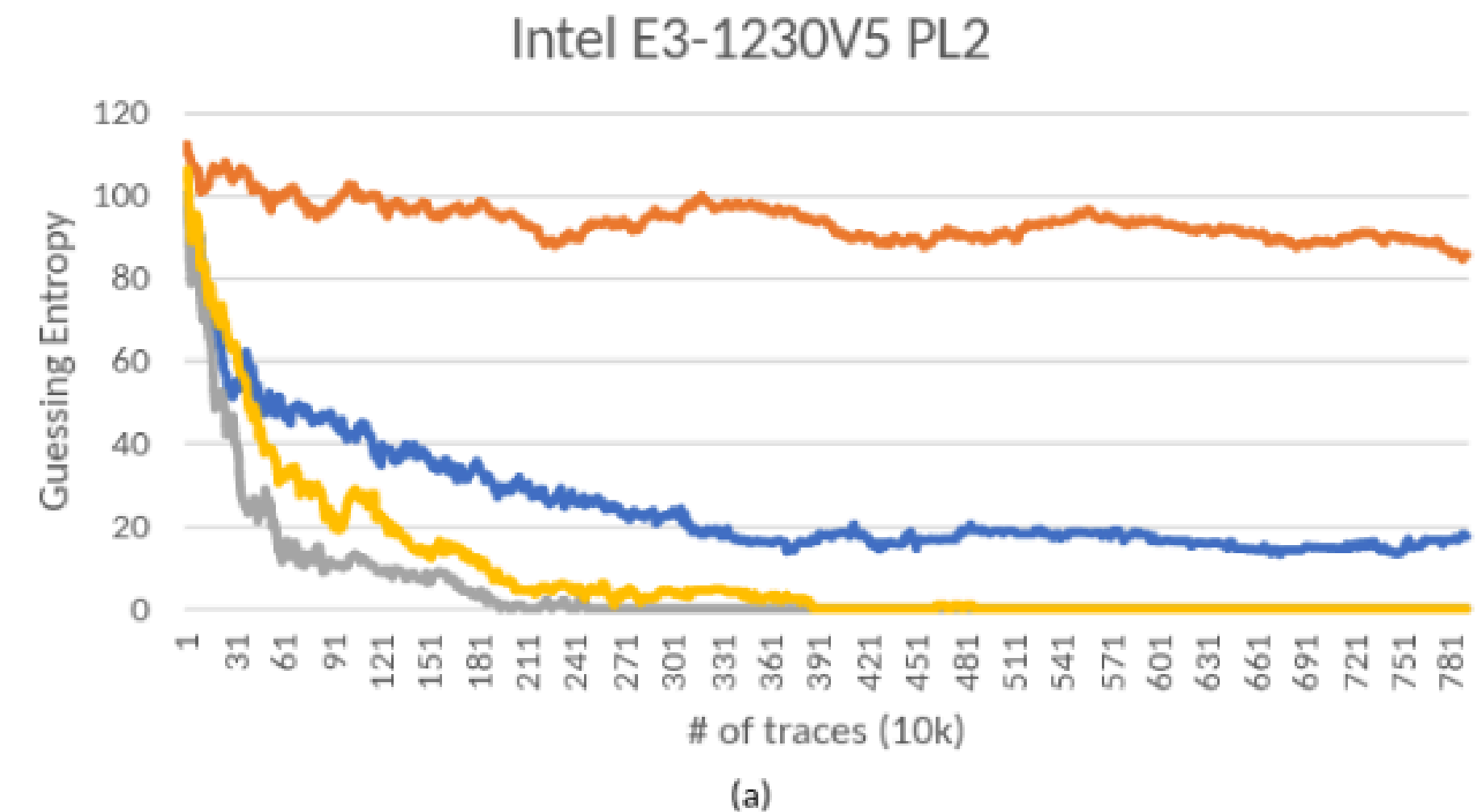
E3-1230V5 **GE 0** Round10-HD

i7-1185G7 **GE 21.4** Round10-
HW+HD

Xeon Gold 6326 **GE 2.6** Round0-HW

AMD Ryzen 5 5600G **GE 1.5** Round10-
HW+HD

On E3-1230V5, Round10-HD reached GE = 0 with
<2M traces — all 16 key bytes recovered.



The End